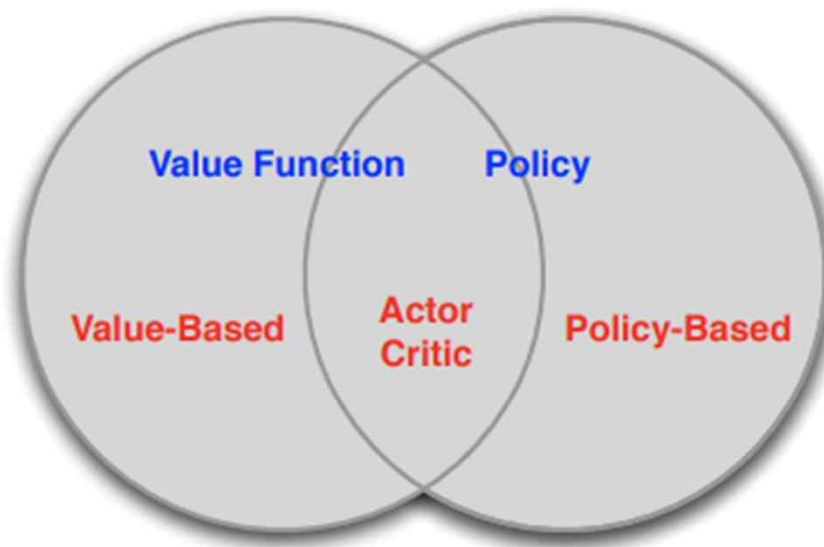
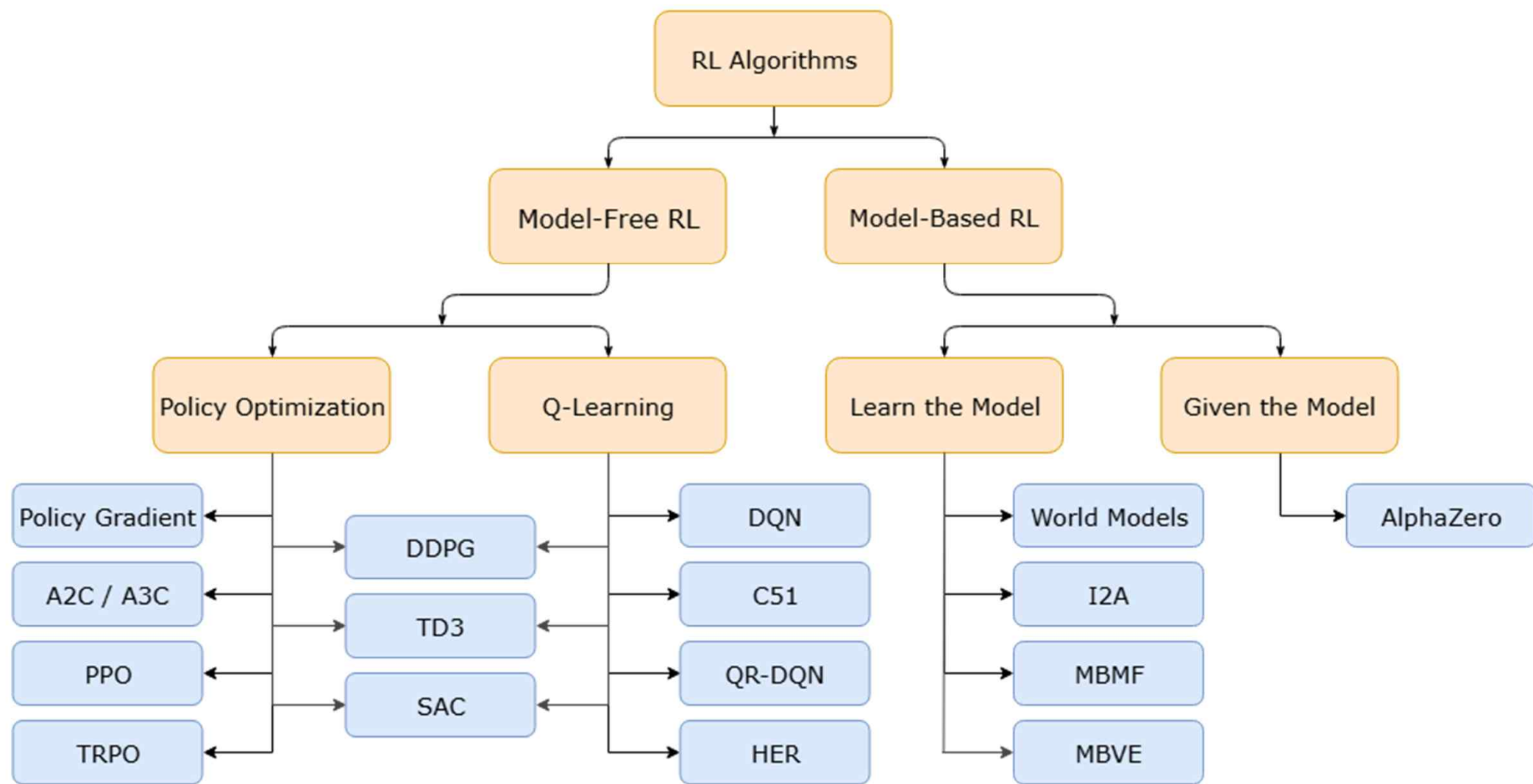


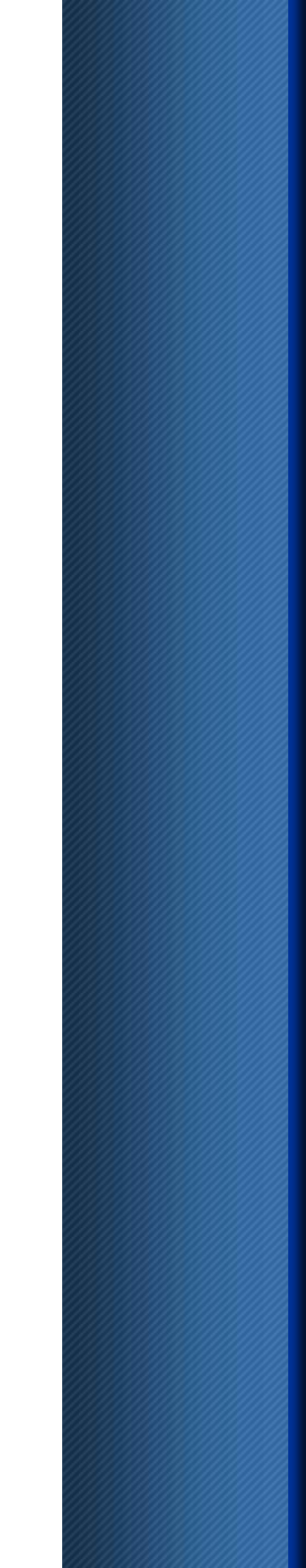
DeepRL

Value-Based and Policy-Based RL

- Value Based
 - Learnt Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learnt Policy
- Actor-Critic
 - Learnt Value Function
 - Learnt Policy







DQN

Generalization with deep learning

- ✓ Value function approximation with deep learning
- ✓ Large scale or infinite dimensional state can be solvable

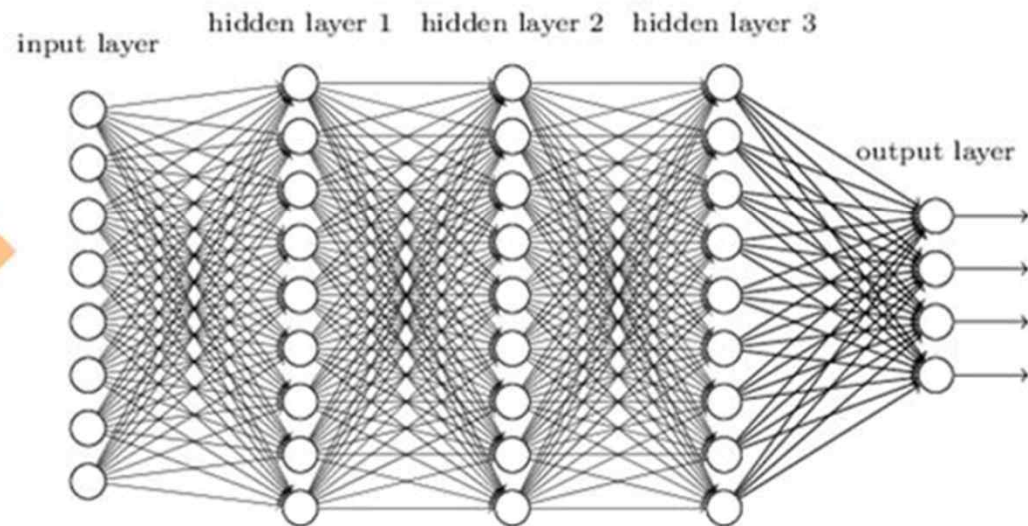
Tabular Q value function

$\hat{Q} =$

state \ action	A	B	C	D	E	F
A	-	-	-	-	80	-
B	-	-	-	64	-	100
C	-	-	-	64	-	-
D	-	80	51	-	80	-
E	64	-	-	64	-	100
F	-	80	-	-	80	100



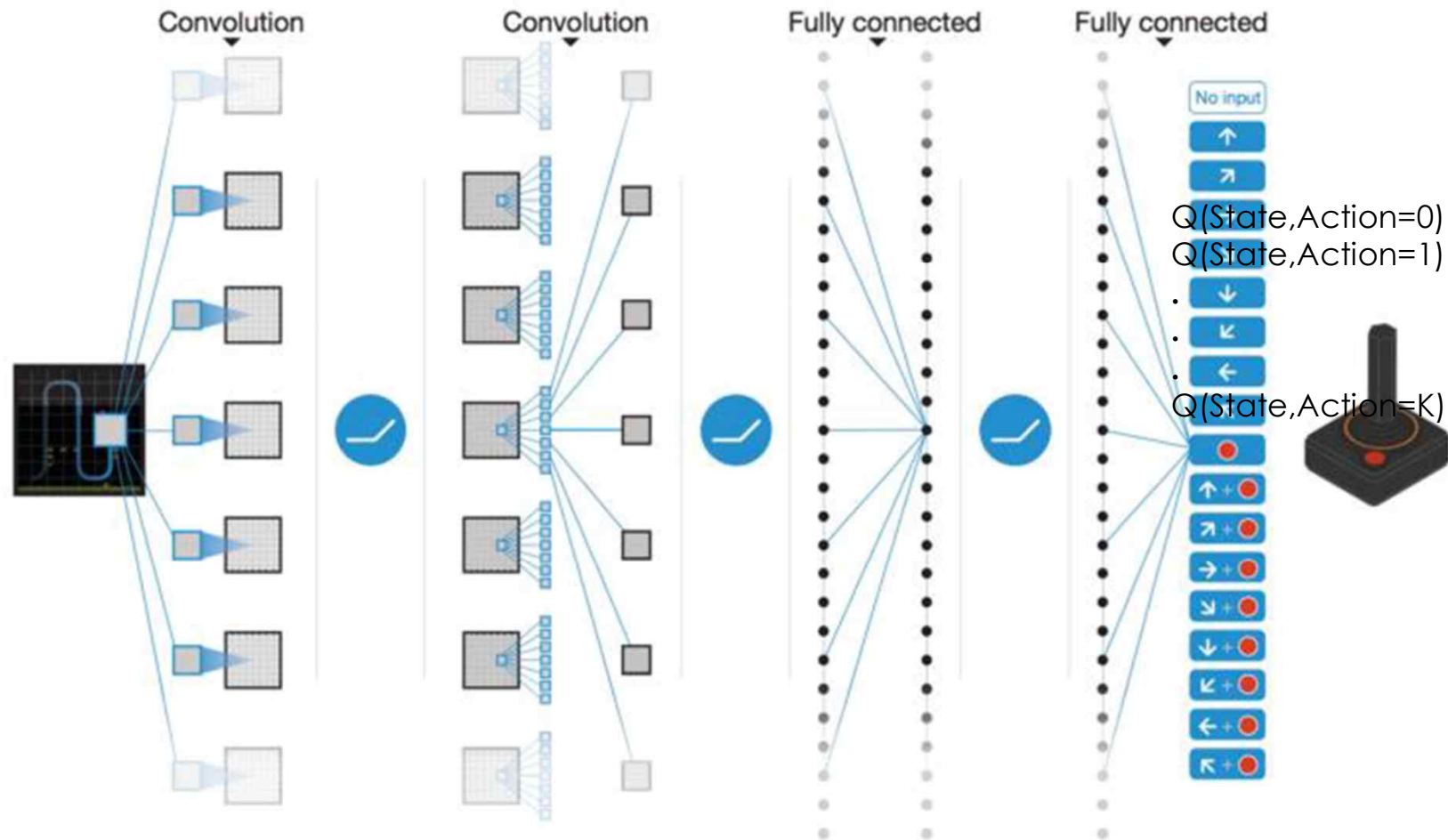
Deep neural network



- ✓ This needs supervised learning techniques and online moving target regression.

Deep Q Networks (DQN)

- ✓ Q learning에서 특정 state에 대한 action-value function으로 CNN을 사용



Loss function

- ✓ Q learning의 업데이트 식

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \overset{\text{TD error}}{\left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]}$$

- ✓ TD error의 크기를 Loss로 사용, regression 방식으로 학습

$$L = \frac{1}{2} \left[\underbrace{r + \max_{a'} Q(s', a')}_{\text{target}} - \underbrace{Q(s, a)}_{\text{prediction}} \right]^2$$

Target Network

$$L = \frac{1}{2} \left[\underbrace{r + \max_{a'} Q(s', a')}_{\text{target}} - \underbrace{Q(s, a)}_{\text{prediction}} \right]^2$$

- ✓ Moving Target Regression : Q를 학습하는 순간, target 값도 같이 변화 학습 성능 저하
- ✓ Target Network : DQN과 동일한 구조를 가지고 있으며 학습 도중 weight값이 변하지 않는 별도의 네트워크로 $Q(s', a')$ 로 고정된 target value를 사용하여 학습 안정성 개선
- ✓ (Target Network의 weight값들은 주기적으로 DQN의 것을 복사)

Replay Memory

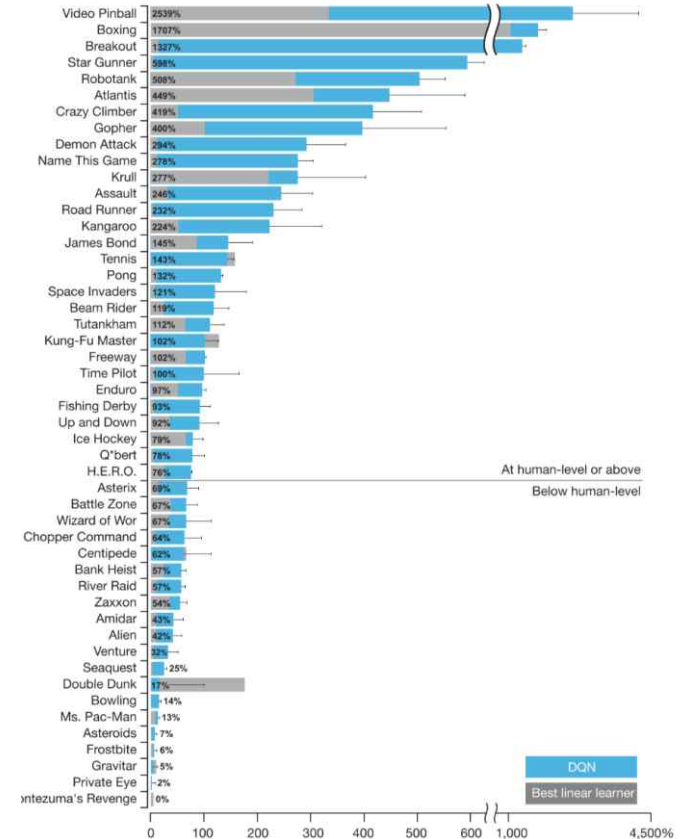
- ✓ **Data inefficiency** : 그 동안 경험한 experience들을 off-policy learning으로 재활용(Mini-batch learning)
- ✓ **Data correlation** : Mini-batch를 On-line으로 구성할 경우 Mini batch 내의 데이터들이 서로 비슷함(Correlated)
 - 한쪽으로 치우친 불균형한 학습이 발생함
- ✓ **Replay Memory** : (State, Action, Reward, Next State) 데이터를 Buffer에 저장해놓고 그 안에서 Random Sampling하여 Mini-batch를 구성하는 방법으로 해결

Reward Clipping

- ✓ **Reward Scale problem** : 도메인에 따라 Reward의 scale이 다르기 때문에 학습되는 Q-value의 크기도 제각각. 이렇게 학습해야하는 Q-value의 variance가 매우 큰 경우 (ex : -1000~1000) 신경망 학습이 어려움.
- ✓ **Reward Clipping**: Reward의 크기를 -1~1 사이로 잘라내어 안정적으로 학습

DQN의 성능

- ✓ •ATARI 2600 고전게임에서 실험
- ✓ •절반 이상의 게임에서 사람보다 우수
- ✓ •기존방식 (linear)에 비해 월등한 향상
- ✓ •일부 게임은 학습에 실패함.
- ✓ (sparse reward, 복잡한 단계를 가진 게임)



DQN의 발전된 모델들 - Double DQN

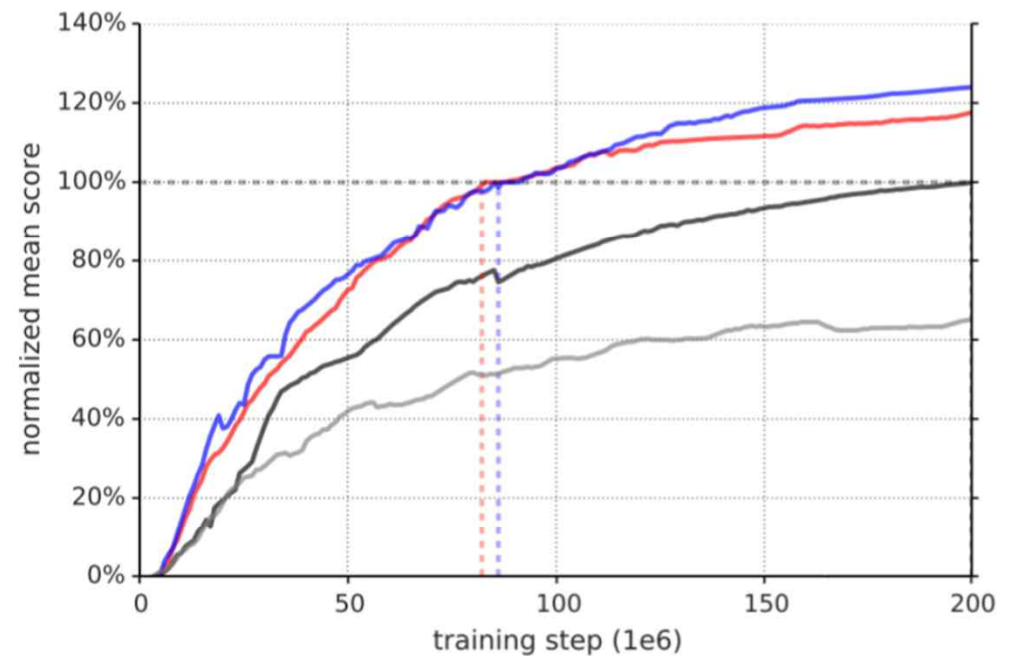
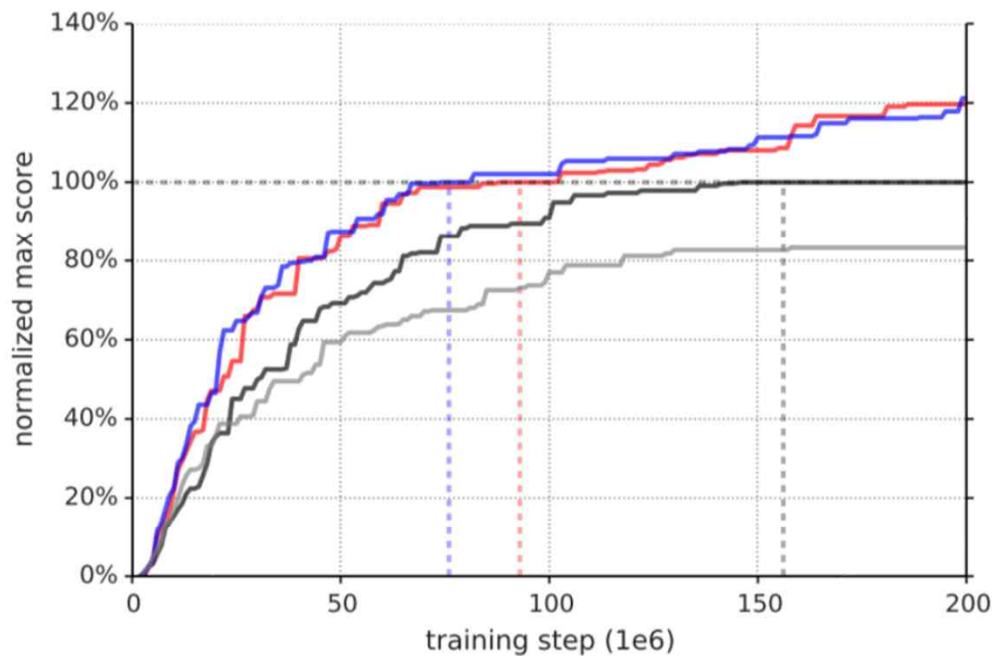
$$Y_t^{\text{DQN}} \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t^-).$$

$$Y_t^{\text{DoubleQ}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \underset{a}{\operatorname{argmax}} Q(S_{t+1}, a; \theta_t); \theta'_t).$$

- ✓ Positive bias: max 연산자에 의해 Q-value를 실제보다 높게 평가하는 경향이 있음
- ✓ Double Q learning을 사용하여 이 현상을 방지, DQN에 보다 빠른 학습이 가능

DQN의 발전된 모델 - Prioritized Replay Memory

- ✓ 일반적인 DQN은 Replay Memory에서 Uniform Sampling
- ✓ TD error가 높은 데이터들이 선택될 확률을 높여, 더 빠르고 효율적인 학습이 가능



— uniform — rank-based — proportional — uniform DQN



상태, 행동, 보상

행동: 제작리, 좌, 우(발사)
 보상: 별들 꺾 때마다 점수를 받으며 위 층의 별들을 꺾수록 더 큰 점수를 받음
 게임 세팅: 1 에피소드에서 에이전트는 5개의 목숨을 가짐
 목표: 1 에피소드 동안 최대의 점수 얻기

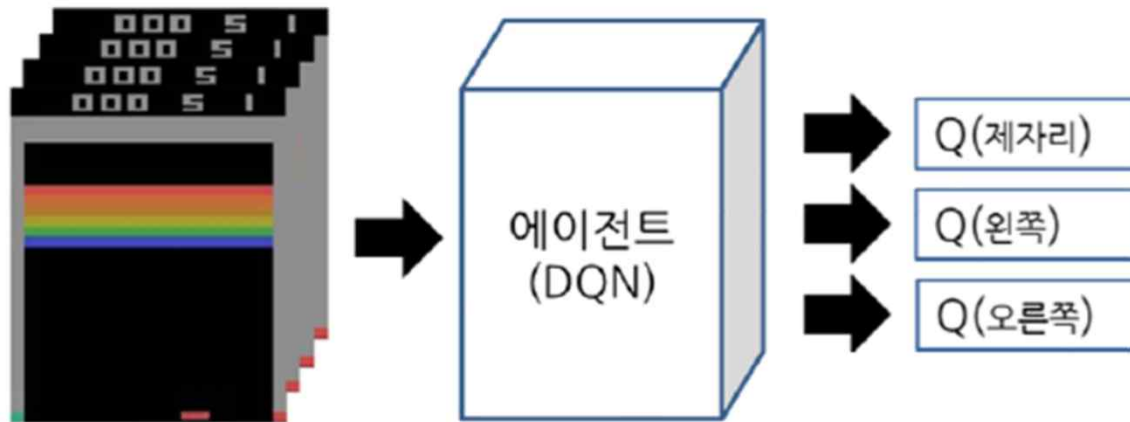


		214	208	64	154	121	122	156	56	135
	22	36	39	170	232	243	139	178	240	82
12	20	18	246	3	91	145	93	118	48	150
22	21	115	99	90	116	1	206	164	49	240
19	69	52	139	142	164	203	14	204	121	42
14	57	160	15	53	134	211	118	27	49	232
16	15	221	237	159	234	162	141	185	235	44
3	86	154	109	145	165	254	25	63	75	75
7	11	140	115	241	202	246	109	102	244	159
83	16	104	148	80	13	241	0	114	12	46
12	20	104	31	239	120	157	243	247	87	123
61	59	60	51	245	224	2	233	62	66	64
15	24	35	48	81	38	18	129	196	20	67
19	22	61	174	84	165	34	24	164	46	106
35	14	108	162	180	95	89	33	123	36	128
21	91	44	147	194	189	43	65	222	205	160
76	23	92	34	27	201	127	25	182	36	14
15	55	16	39	83	195	88	135	186	96	161
10	20	109	53	119	15	199	58	30	94	129
1	24	21	123	213	132	104	151	178	73	
52	70	98	195	236	176	234	194	225		

브레이크아웃의 상태는 RGB 이미지
 모든 상태의 큐합수를 다 저장하고 업데이트하는 방식으로는 불가능

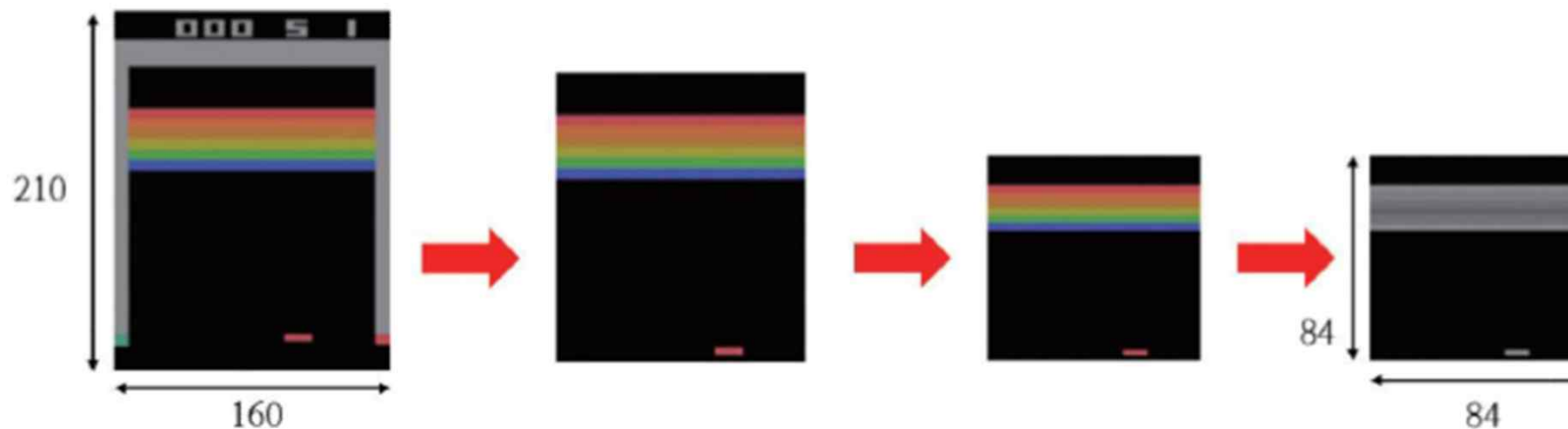


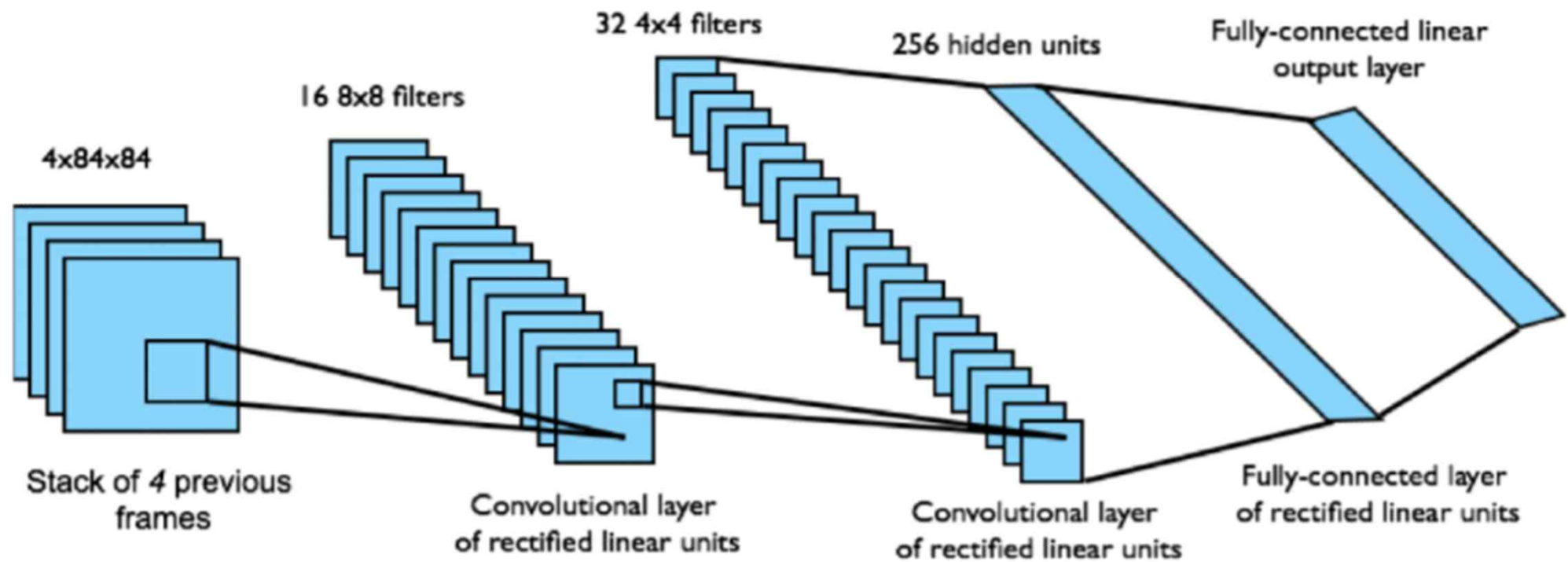
브레이크아웃의 상태는 RGB 이미지 4장으로 이루어진 히스토리



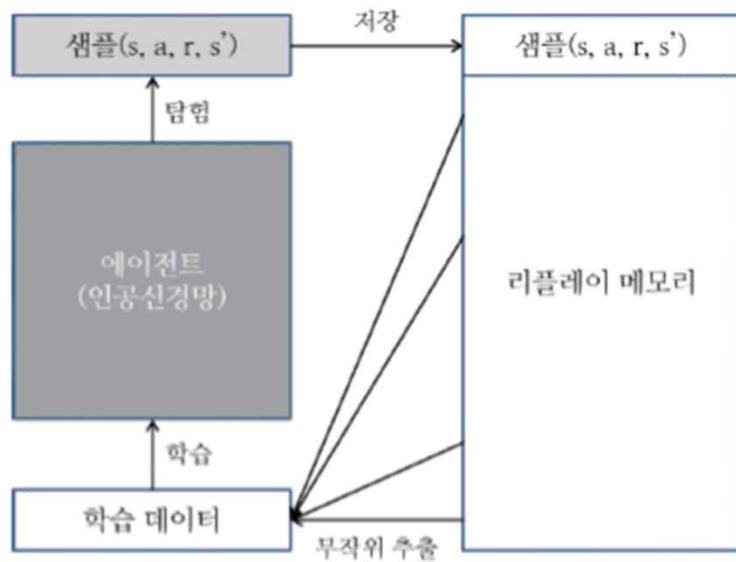
큐함수를 인공지능망으로 근사
상태(히스토리) -> 큐-신경망 -> 각 행동에 대한 큐함수 값

브레이크아웃 이미지의 전처리 과정(Rescale -> grayscale)

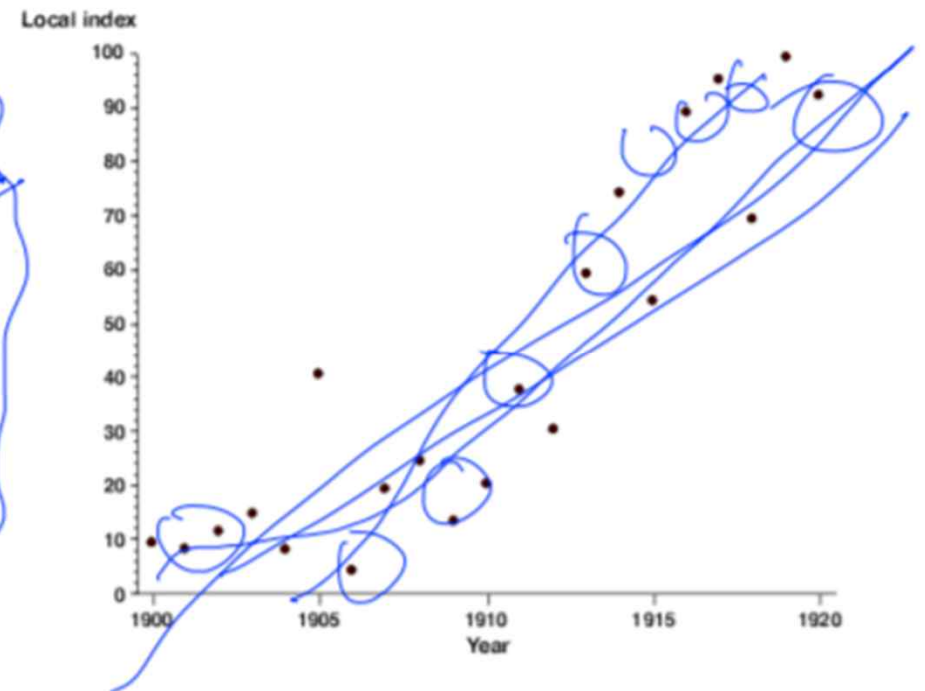




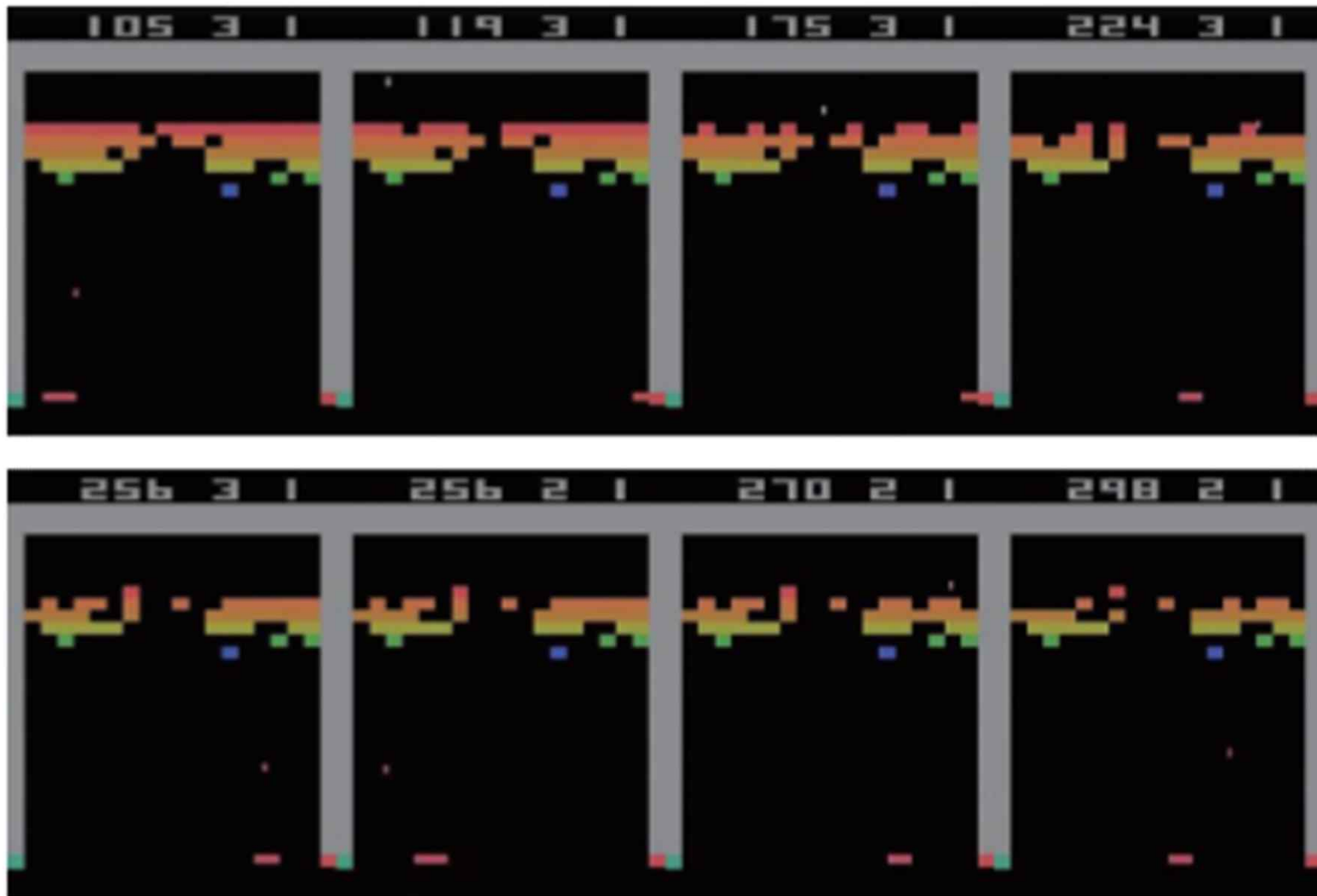
ATARI

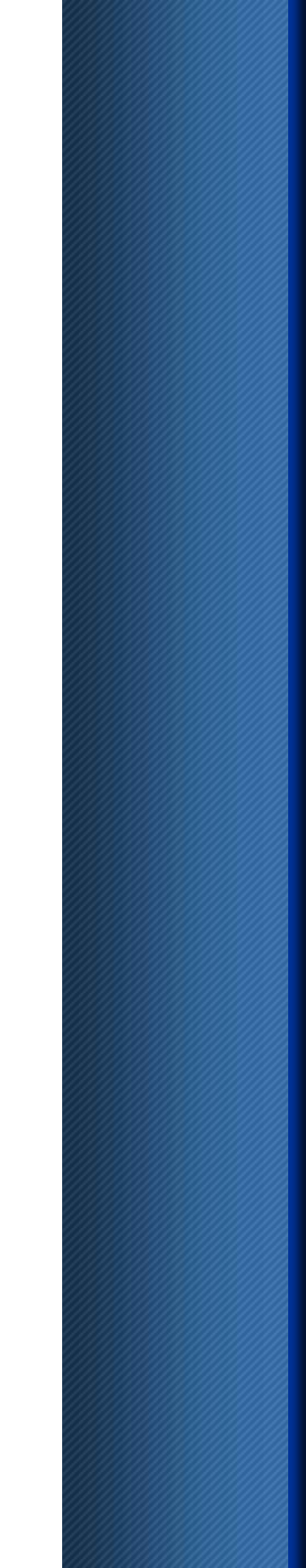


s_1, a_1, r_2, s_2
s_2, a_2, r_3, s_3
s_3, a_3, r_4, s_4
...
$s_t, a_t, r_{t+1}, s_{t+1}$



터널을 뚫어서 점수를 얻는 정책을 학습





Policy Gradient

DQN이 이론 점과 부족한 점

✓ 1. DQN01 이론 점

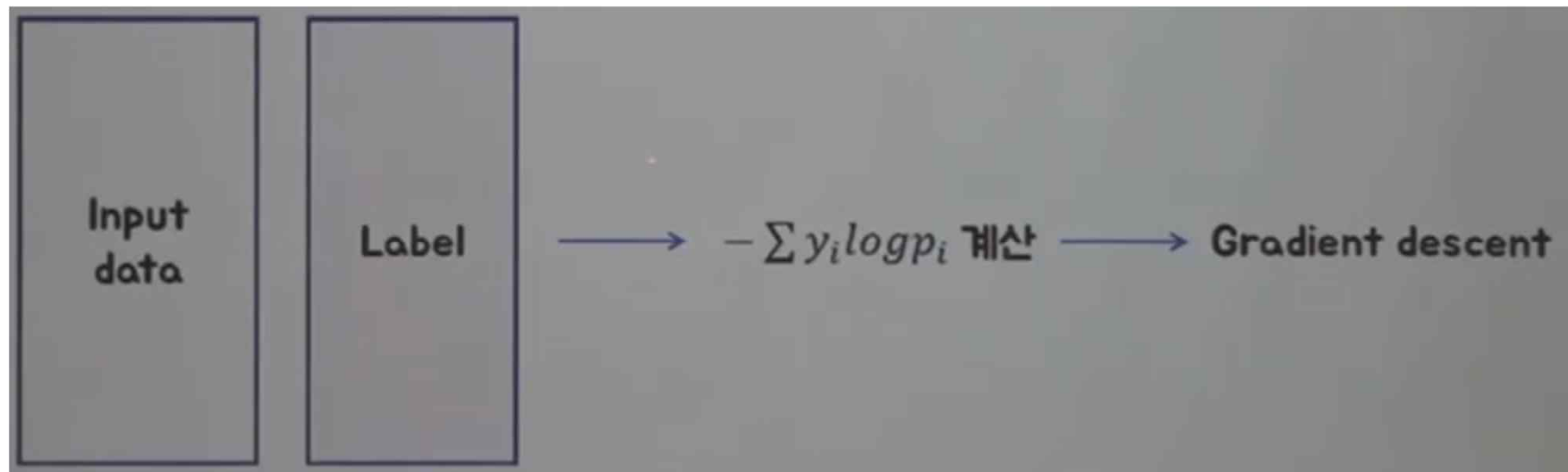
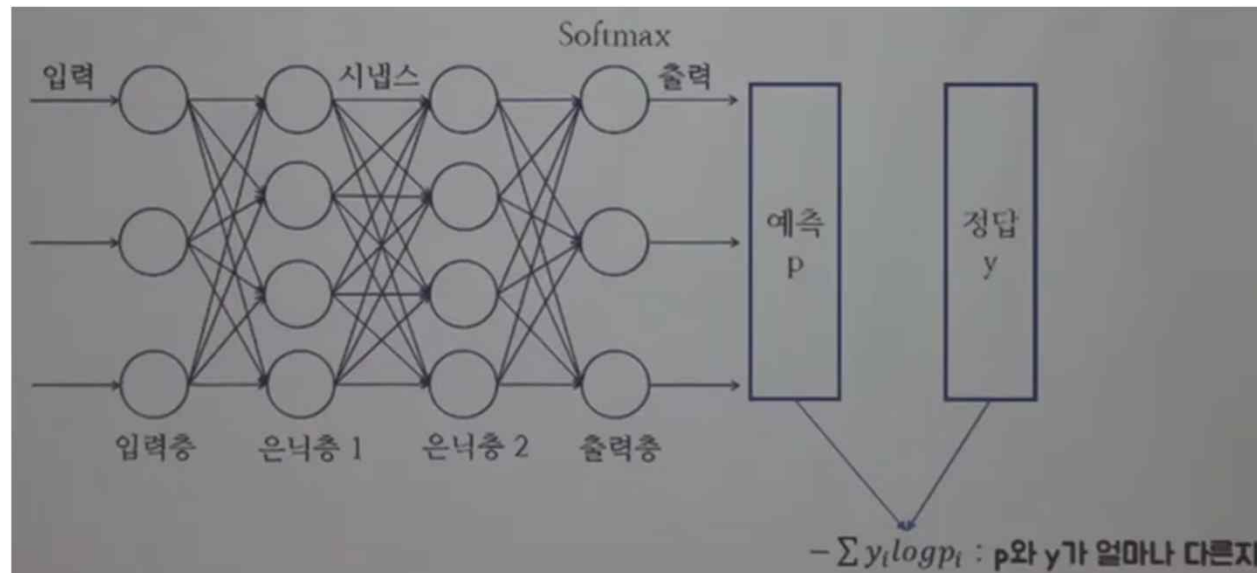
- 게임 화면을 상태 입력으로 받아서 학습(CNN)
- 사람보다 게임 플레이를 더 잘하는 에이전트
- 샘플들 사이의 강한 상관관계를 리플레이 메모리로 해결

✓ 2. DQN01 부족한 점

- 많은 메모리의 사용
- 느린 학습 속도
- 불안정한 학습 과정(가치함수에 대한 greedy policy이므로)

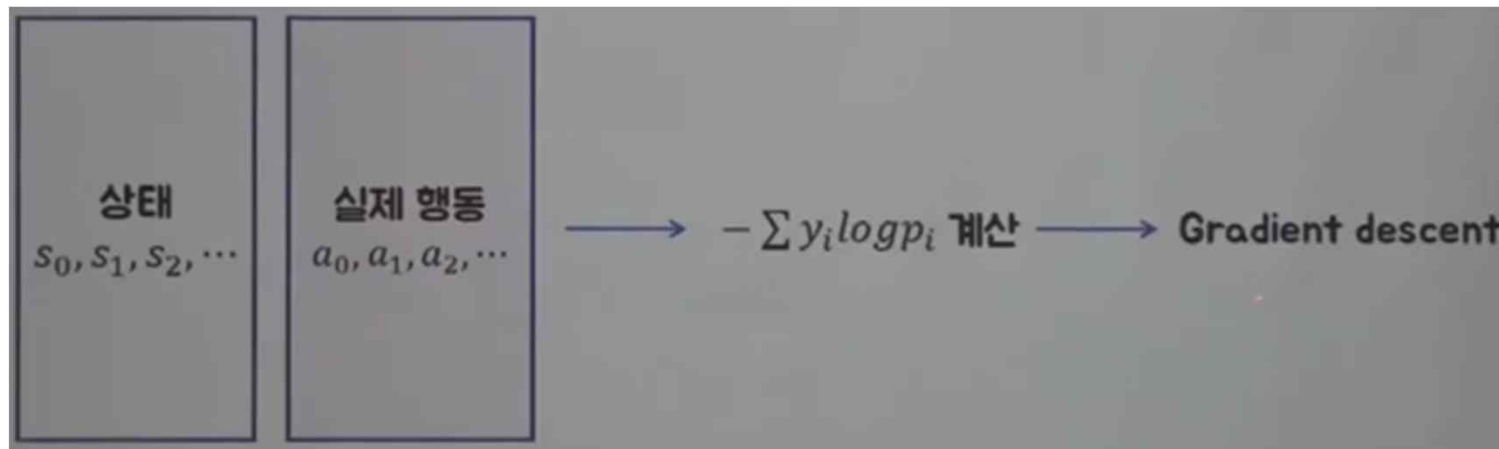
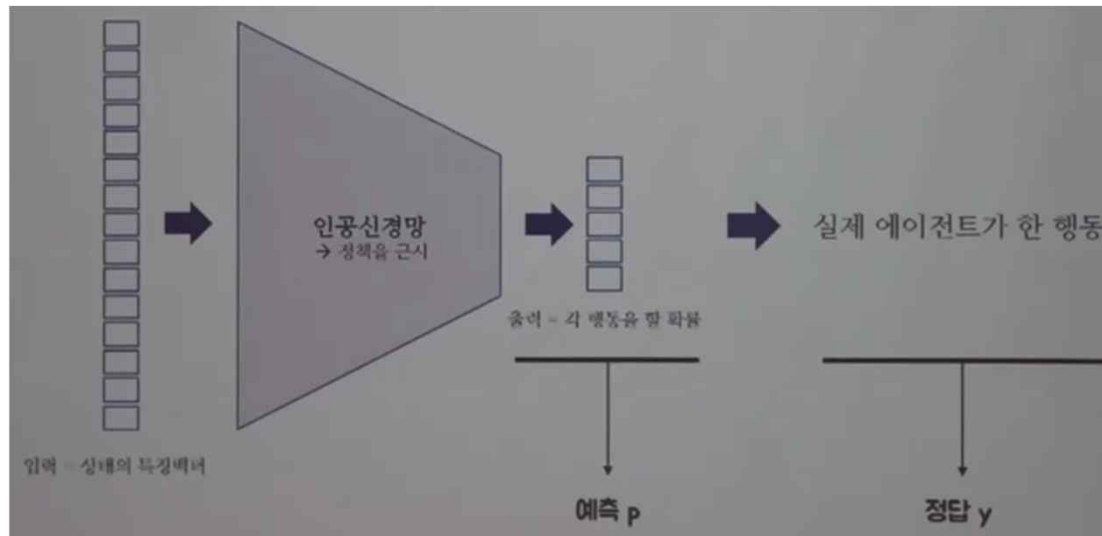
지도학습 Cross entropy

✓ 지도학습의 크로스 엔트로피(Cross entropy)



강화학습 Cross entropy

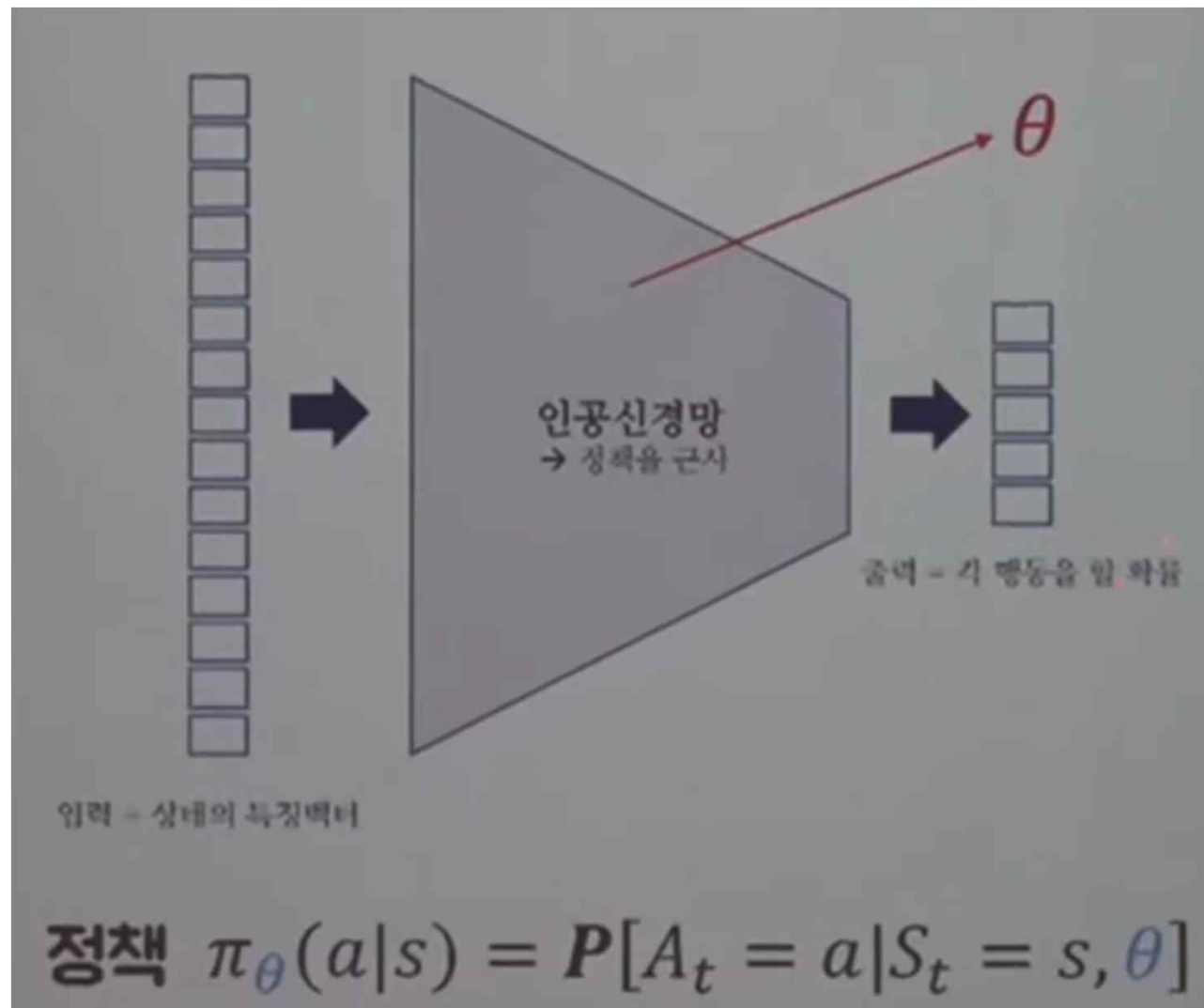
✓ 강화학습의 크로스 엔트로피(Cross entropy)



나의 예측을 실제 행동에 가깝게 만든다 → 어떤 의미 ??
업데이트의 방향성이 중요

정책의 근사화

- ✓ 인공지능망으로 정책을 근사 → 정책 신경망



목표함수 요약

1. 경로(Trajectory)의 개념

- 정책 π_θ 에 따라 에이전트가 환경과 상호작용하며 생성되는 상태·행동·보상의 기록
- 경로 예시

$$\tau = s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T$$

2. 목표함수 $J(\theta)$

- 에이전트가 정책을 따를 때 기대되는 총 보상
- 즉, "이 정책이 얼마나 좋은가?"를 수치로 표현한 함수

$$J(\theta) = \mathbb{E} \left[\sum_{t=0}^{T-1} r_{t+1} \mid \pi_\theta \right] = \mathbb{E}[r_1 + r_2 + r_3 + \dots + r_T \mid \pi_\theta]$$

3. 핵심 요약

- 정책이 바뀌면 경로가 바뀌고, 경로가 바뀌면 보상도 달라진다.
- 강화학습의 목표는
정책 파라미터 θ 를 조정하여 $J(\theta)$ 를 최대화하는 것.

Gradient ascent

Gradient Ascent에 의한 정책 업데이트

1. 목표

- 목표 함수 $J(\theta)$ 를 최대화하기 위해 정책 파라미터 θ 를 업데이트한다.

2. 핵심 아이디어

- $J(\theta)$ 가 증가하는 방향(경사 방향)으로 θ 를 조정한다.
- 즉, 경사 상승(Gradient Ascent) 방식으로 정책을 개선한다.

3. 업데이트 공식

$$\theta' = \theta + \alpha \nabla_{\theta} J(\theta)$$

$$\nabla_{\theta} J(\theta) = \text{Policy gradient}$$

- α : 학습률
- $\nabla_{\theta} J(\theta)$: Policy Gradient

4. 핵심 요약

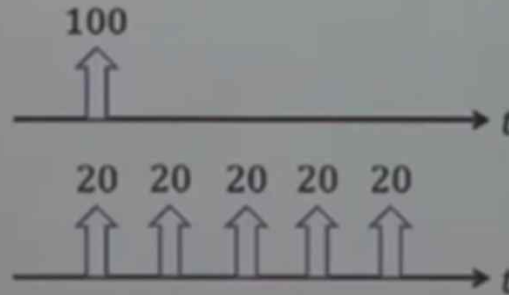
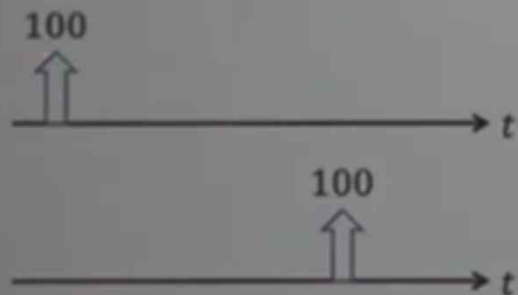
- Policy Gradient는
“보상이 높은 방향으로 정책을 개선하는 변화량”
을 의미한다.

REINFORCE

REINFORCE = 몬테카를로 Policy gradient

$$\nabla_{\theta} J(\theta) = E_{\tau} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t+1}^T r_{t'} \right]$$

$\sum_{t'=t}^{T-1} r_{t'}$ 와 같은 단순 보상의 합 $\rightarrow$ 3가지 문제



$$0.1 + 0.1 + \dots = \infty$$
$$1 + 1 + \dots = \infty$$

감가율(discount factor) $0 \leq \gamma \leq 1$ 의 도입

REINFORCE

1. 한 에피소드를 현재 정책에 따라 실행

2. Trajectory를 기록

3. 에피소드가 끝난 뒤 G_t 계산

4. Policy gradient를 계산해서 정책 업데이트 Policy gradient = $\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$

5. (1~4) 반복

에피소드 마다 업데이트 → 몬테카를로 Policy gradient = REINFORCE

REINFORCE

1. 한 에피소드를 현재 정책에 따라 실행
2. Trajectory를 기록
3. 에피소드가 끝난 뒤 G_t 계산
4. Policy gradient를 계산해서 정책 업데이트 $\text{Policy gradient} = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$
5. (1~4) 반복

에피소드 마다 업데이트 → 몬테카를로 Policy gradient = REINFORCE

REINFORCE의 문제

1. Variance가 높다
2. 에피소드마다 업데이트가 가능하다(on-line X)

몬테카를로 → TD(Temporal-Difference)

REINFORCE → Actor-Critic

Actor-Critic

Actor-Critic = REINFORCE + 실시간 학습

$$\nabla_{\theta} J(\theta) = E_{\tau} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_{\pi_{\theta}}(s_t, a_t) \right]$$
$$\sim \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_{\pi_{\theta}}(s_t, a_t)$$

만약 $Q_{\pi_{\theta}}(s_t, a_t)$ 를 알 수 있다면 매 time-step마다 업데이트하는 것이 가능!!

$Q_{\pi_{\theta}}(s_t, a_t) \sim Q_w(s_t, a_t)$ Let's make Critic!!!!

$$\nabla_{\theta} J(\theta) \sim \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_w(s_t, a_t)$$

Advantage 함수 = 큐함수 - 베이스라인 $\rightarrow$ Variance를 낮춰준다

큐함수 : 특정 상태, 특정 행동에 따른 값

가치함수 : 특정 상태, 전반적 행동에 따른 값 $\rightarrow$ 베이스라인

$$\nabla_{\theta} J(\theta) \sim \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q_w(s_t, a_t) - V_v(s_t))$$

Actor-Critic

$$\nabla_{\theta} J(\theta) \sim \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q_w(s_t, a_t) - V_v(s_t))$$

Q와 V를 둘 다 근사하는 건(ex 뉴럴넷) 비효율적

$Q(s_t, a_t) = E[r_{t+1} + \gamma V(s_{t+1}) | s_t, a_t]$ 을 이용!

$$\nabla_{\theta} J(\theta) \sim \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (\underbrace{r_{t+1} + \gamma V_v(s_{t+1})}_{\text{큐함수}} - \underbrace{V_v(s_t)}_{\text{베이스라인}})$$

큐함수

베이스라인

1. Actor

1) 정책을 근사: θ

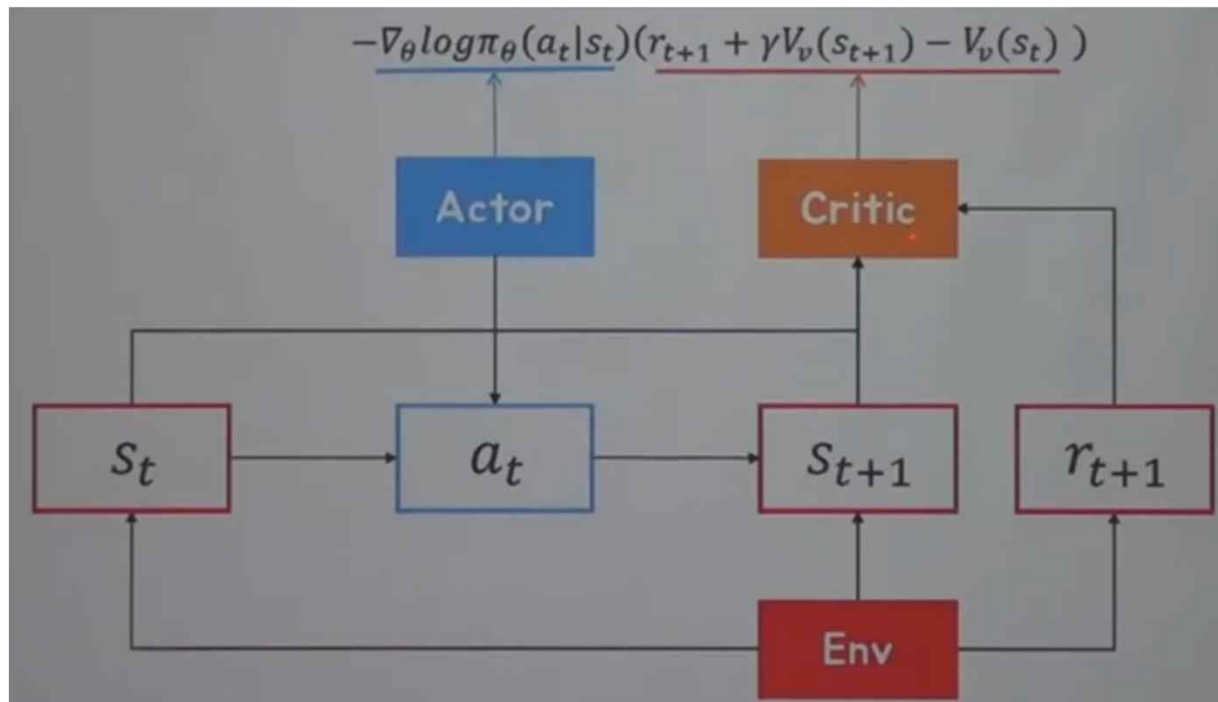
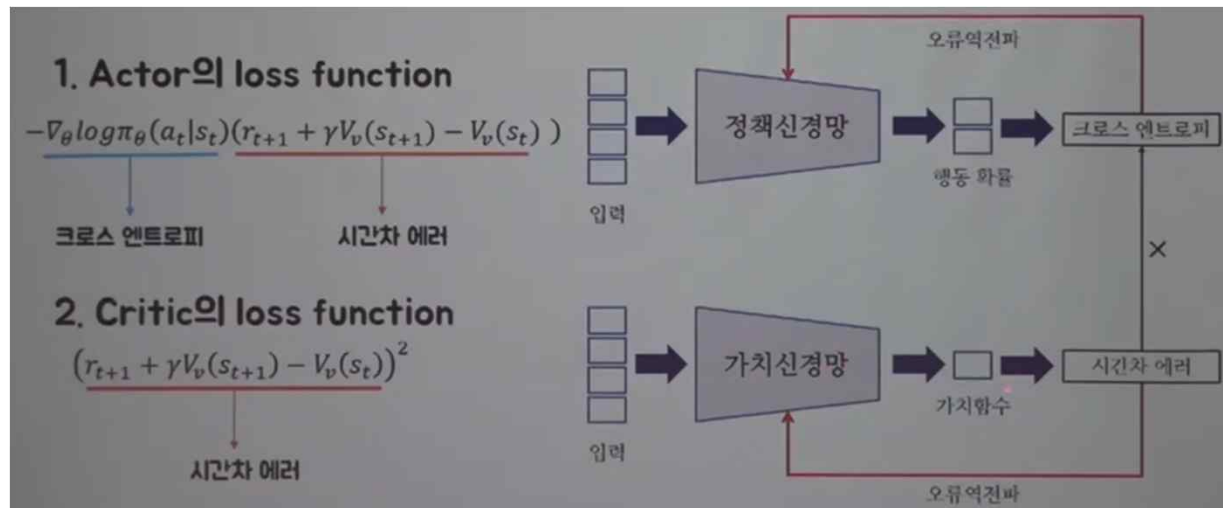
2) $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (r_{t+1} + \gamma V_v(s_{t+1}) - V_v(s_t))$ 로 업데이트

2. Critic

1) 가치함수(Value function)을 근사: v

2) $(r_{t+1} + \gamma V_v(s_{t+1}) - V_v(s_t))^2$ 의 오차함수로 업데이트

Actor-Critic



Idea

TRPO (Trust Region Policy Optimization)

- 기존 문제: “Policy를 너무 멀리 업데이트하면 성능이 내려간다”
- Trust-region = 신뢰할 수 있는 작은 영역 안에서만 업데이트.

PPO

- TRPO의 안정성 유지. 구현을 단순화함.

Main components

Loss

1. Clipped surrogate objective
2. Adaptive KL penalty

- **Constrained Form**

$$\begin{aligned} & \underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \\ & \text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta. \end{aligned}$$

- **Penalized Form**

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\underbrace{\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}}_{\substack{\downarrow \\ r_t(\theta)}} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right]$$
$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad \text{so } r(\theta_{\text{old}}) = 1.$$

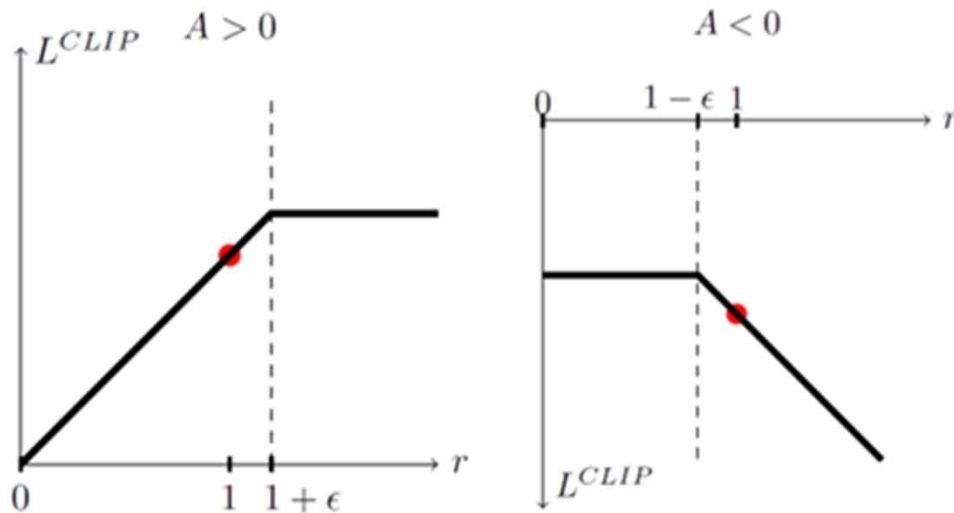
Probability ratio

새 정책이 옛 정책 대비 얼마나 행동 확률을 늘리거나 줄였는가

Clipped Surrogate Objective

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(\underbrace{r_t(\theta)}_{\text{Original Loss}}, \underbrace{\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)}_{\text{Clipped Loss}} \hat{A}_t) \right]$$

Lower bound of unclipped objective = L^{CPI}



- θ_{old} 근처에서는 (즉, $r=1$ 근처) L^{CPI} 와 L^{CLIP} 가 1차 근사.

SAC까지의 전체 진화 흐름 요약

모델	개선 내용	SAC에 미친 영향
REINFORCE	최초의 Policy Gradient	Actor 구조의 기본
Actor-Critic	Baseline으로 분산 감소	Actor + Critic 틀 확립
DDPG	Off-policy + 연속제어	Replay Buffer, Target Net
TRPO	안정적 정책 업데이트	안정성 개념
PPO	간단한 stable 업데이트	정책 업데이트 안정성
TD3	Over-estimation 해결	Twin Q, smoothing 기법
SAC	Max Entropy, α 자동 조절, Off-policy 최강	최종 통합형 모델

- ✓ 결론적으로 SAC는 TD3의 안정성 + PPO/TRPO의 안정적 업데이트 철학 + DDPG의 Off-policy 효율 + 엔트로피 기반 탐색 을 모두 통합한 강력한 강화학습 모델입니다.



감사합니다.

e2g1234@naver.com
